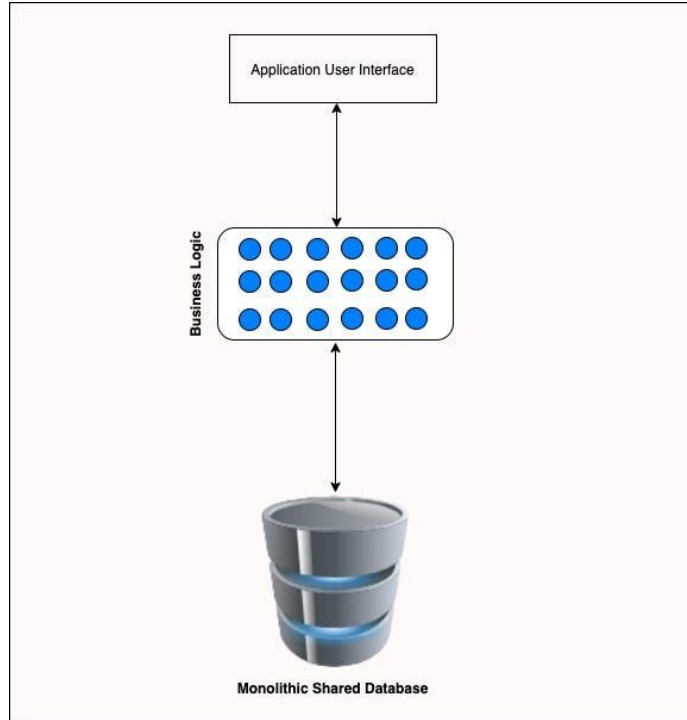


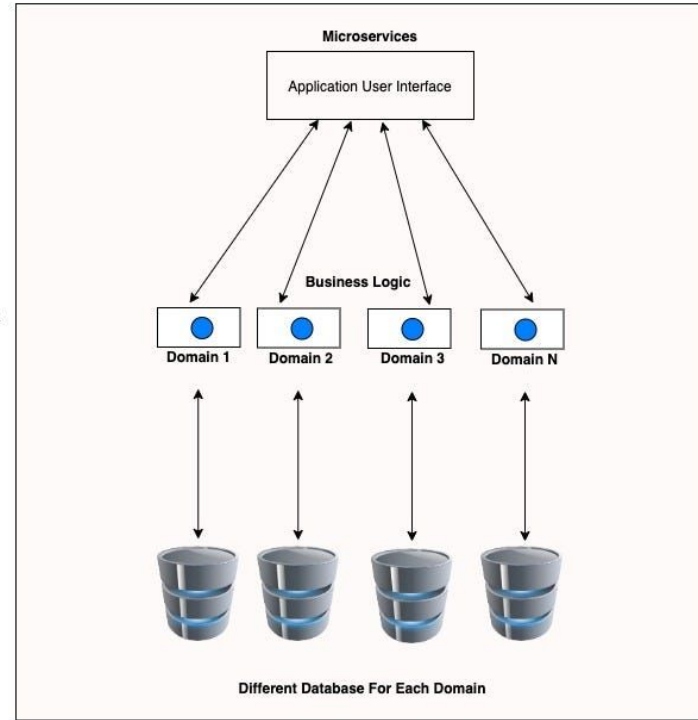
BDD en Microservices



Récapitulatif



OR



Comparaison avec le design monolithique

	Monolithique	Microservices
Pros	Simplicité Fiabilité ACID* compliance	Couplage faible Evolution de schéma, mise à l'échelle et optimisation indépendante Eventail de choix
Cons	Evolutivité limitée Point de défaillance unique Goulots d'étranglement des performances Latence pour les applications globales	Pas de compliance ACID Complexité opérationnelle accrue Nécessité d'intégration de modèles (événements, réplication, caches)

*ACID : atomicité, cohérence, isolement et durabilité



Stratégies de conception de schémas

- Utilisez les **Universally Unique Identifier** (abrégé UUID) comme identifiants stables entre les services.
- Conservez les foreign keys locales au service.
- Privilégiez les agrégats et la dénormalisation lorsque cela améliore les performances et l'indépendance.
- Maintenez des modèles de lecture ou des vues matérialisées pour les requêtes nécessitant des données inter-services.

Exemple :

Le service Commande gère les commandes des utilisateurs, le service Inventaire stocke les quantités de produits. La table Commande contient product_id (UUID) mais ne référence pas directement la table Inventaire.



Stratégies de conception de schémas

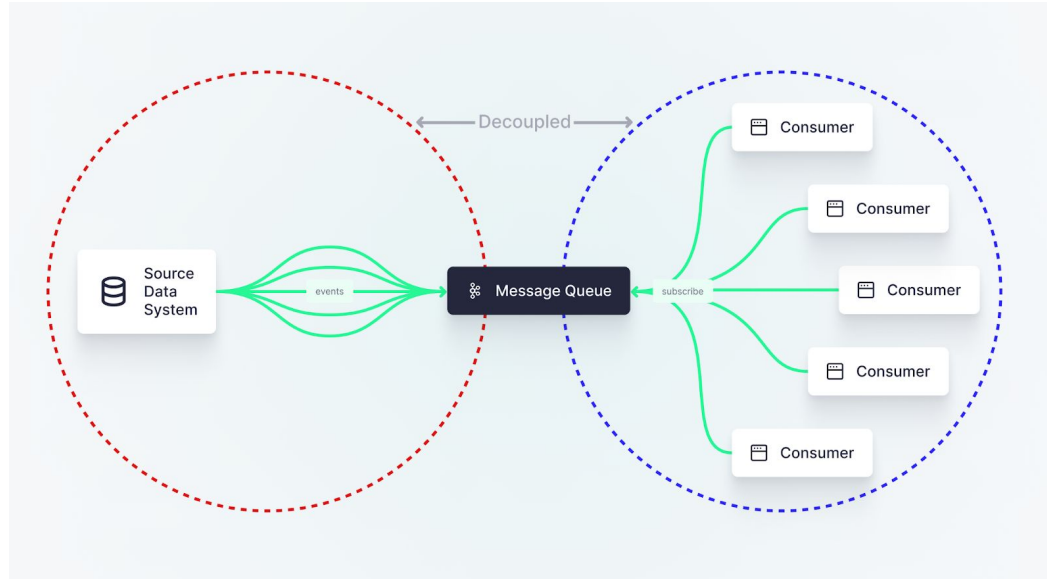
- Utilisez les **Universally Unique Identifier** (abrégé UUID) comme identifiants stables entre les services.
- Conservez les foreign keys locales au service.
- Privilégiez les agrégats et la dénormalisation lorsque cela améliore les performances et l'indépendance.
- Maintenez des modèles de lecture ou des vues matérialisées pour les requêtes nécessitant des données inter-services.

Exemple :

Le service Commande gère les commandes des utilisateurs, le service Inventaire stocke les quantités de produits. La table Commande contient product_id (UUID) mais ne référence pas directement la table Inventaire.



Modèles de cohérence éventuelle



Une application tierce (le plus souvent, un message broker) fera le lien entre vos BDD et vos applications.



L'outboxing

Problème :

Une commande de service doit généralement créer, mettre à jour ou supprimer des agrégats dans la base de données et envoyer des messages ou des événements à un courtier de messages. Par exemple, un service participant à une saga doit mettre à jour des entités métier et envoyer des messages ou des événements. De même, un service publiant un événement de domaine doit mettre à jour un agrégat et publier l'événement.

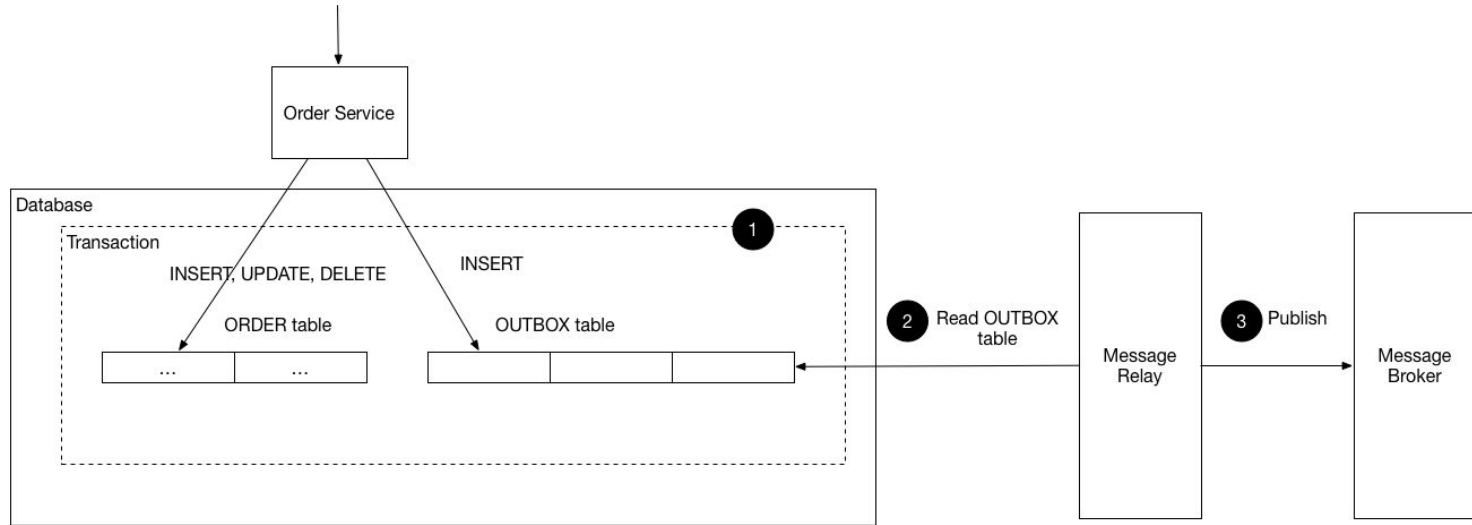
La commande doit mettre à jour la base de données et envoyer les messages de manière atomique afin d'éviter les incohérences de données et les bogues. Cependant, l'utilisation d'une transaction distribuée traditionnelle (2PC) impliquant la base de données et le message broker n'est pas envisageable. En effet, la base de données et/ou le message broker peuvent ne pas prendre en charge les transactions 2PC. Et même s'ils le font, il est souvent préférable d'éviter de coupler le service à la fois à la base de données et au message broker.



L'outboxing

Solution :

La solution consiste à ce que le service émetteur du message le stocke d'abord dans la base de données dans le cadre de la transaction de mise à jour des entités métier. Un processus distinct transmet ensuite les messages au message broker.



Modèle de conception Saga

Objectif :

Réaliser un workflow multiservices se comportant comme une transaction logique (ex. : passer une commande → réserver du stock → débiter le paiement).

Sagas :

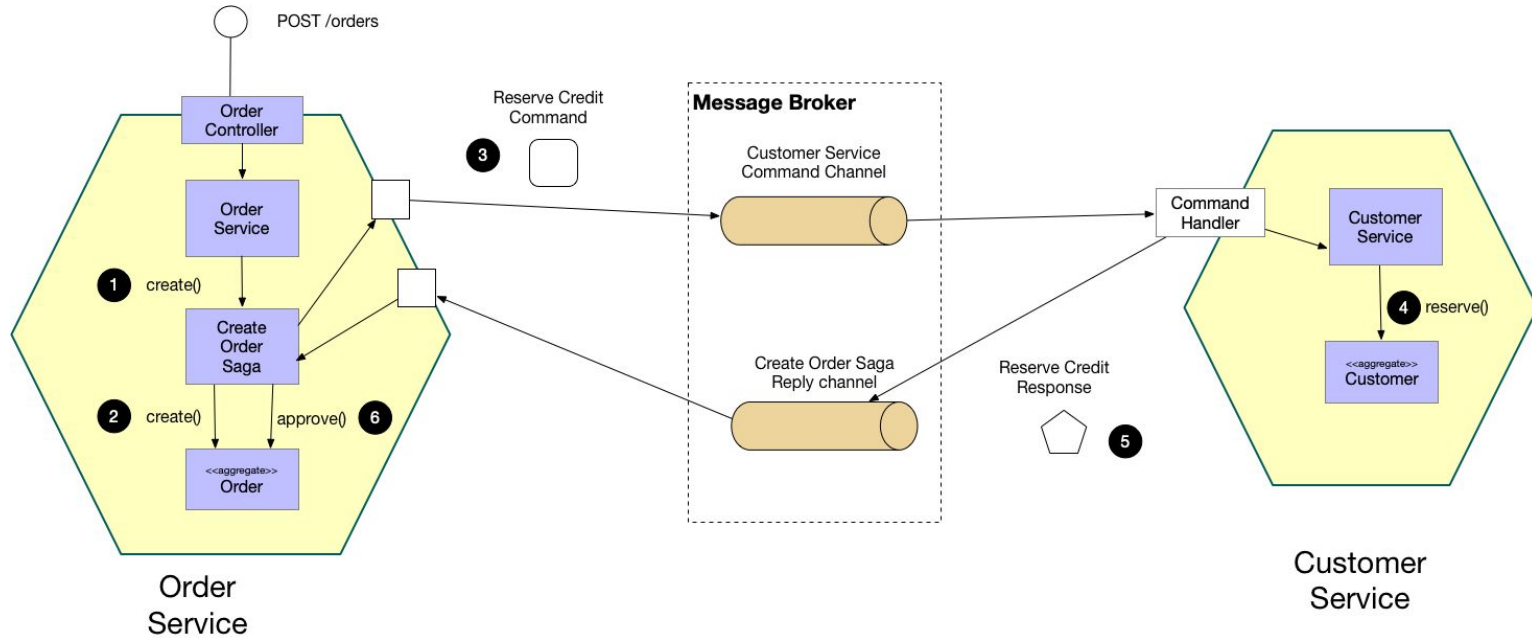
- Orchestration: Un orchestrateur central est responsable de la gestion de la transaction globale.
- Chorégraphie: Chaque service sait quel service appeler ensuite, éliminant le besoin d'un orchestrateur central.

Exemple : Soit la saga PlaceOrder (étapes + compensations en cas d'échec) :

- Le service OrderService crée une commande (en attente) et envoie OrderCreated.
- Le service InventoryService réserve le stock → émet InventoryReserved ou ReservationFailed.
- Le service PaymentService débite la carte → émet PaymentSucceeded ou PaymentFailed.
- En cas d'échec, effectuer les compensations suivantes : libérer le stock, annuler la commande, rembourser le paiement.



Modèle de conception Saga



Attention à la persistance des base de données !



Ne stockez jamais une quelconque forme de donnée utilisateur dans votre code en tant que variable, et SURTOUT les ICC mettez vos bases en dehors des conteneurs docker. Un message broker n'est PAS une BDD !

